Production AI Architecture

Evaluation, Observability, and Safety

Engineering Case Study

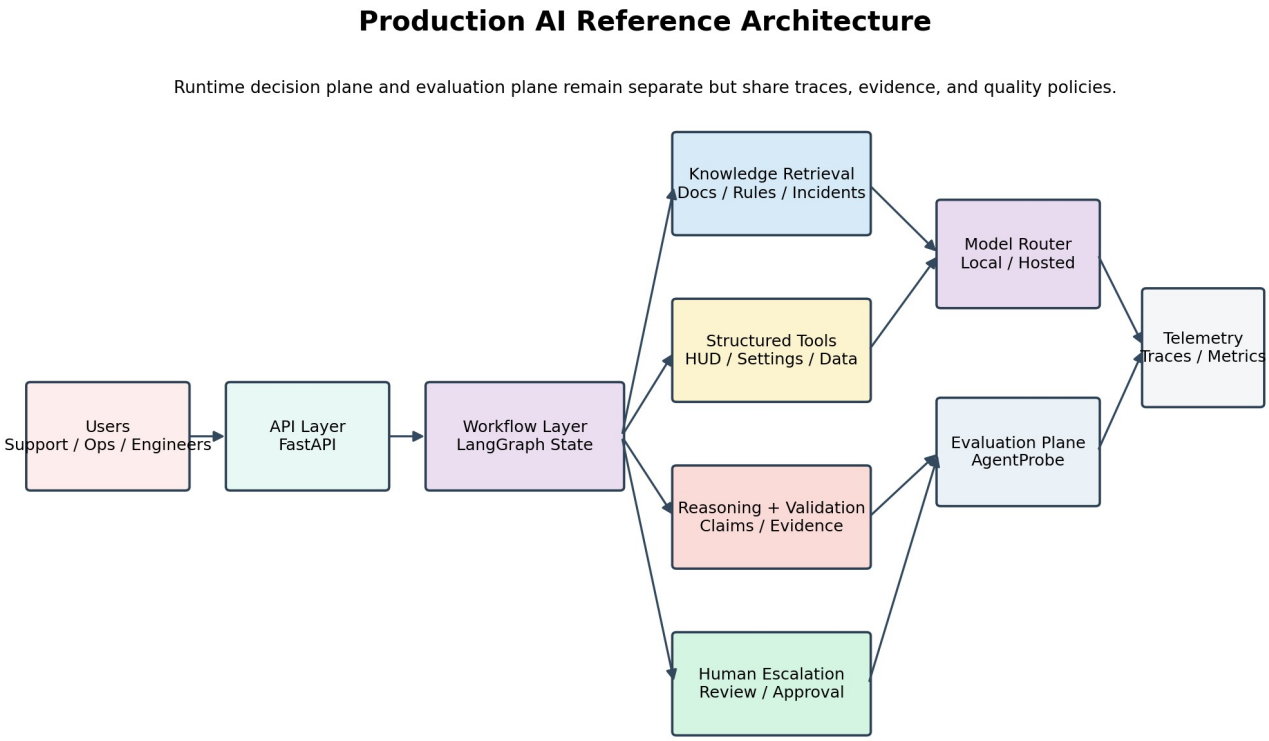


Figure 1. Reference architecture integrating runtime AI workflows with a separate evaluation plane.

Author	Venkata Lakshmi Parimala Pasupuleti
Scope	Cross-project technical architecture derived from LeaseI and AgentProbe
Primary Purpose	Design reliable, evidence-grounded enterprise AI systems and prevent quality regressions
Core Stack	FastAPI, LangGraph, Python tools, retrieval, structured data, AgentProbe, model APIs or local models
Safety Position	Read-only diagnosis; deterministic systems and human reviewers retain authority
Status Discipline	LeaseI built state, documented AgentProbe capabilities, and production targets are explicitly separated

Document Map

1. Executive Summary
2. Source Basis and Status Boundaries
3. Production AI Operating Problem
4. Engineering Principles
5. Reference Architecture
6. Request Lifecycle and Workflow State
7. Knowledge Retrieval Architecture
8. Structured Tool Execution
9. Reasoning and Evidence Validation
10. Safety, Human Review, and Escalation
11. Model Routing and Prompt Contracts
12. Evaluation Strategy
13. Benchmark and Test-Case Design
14. RAG Evaluation and Failure Localization
15. Agent Workflow Evaluation
16. Regression Gates and Release Policy
17. Observability and Distributed Tracing
18. Reliability and Failure Handling
19. Security, Privacy, and Access Control
20. Performance, Capacity, and Cost
21. Storage and Audit Model
22. API and Integration Design
23. Deployment and Environment Strategy
24. Operations and Incident Response
25. Production Feedback Loop
26. Design Decisions and Trade-offs
27. Current Gaps and Modernization Roadmap
28. Lessons Learned
29. Capability Matrix and Conclusion
30. Appendix: Quality Policies and Operational Checklists

GROUNDING	This document combines two project sources: the uploaded LeaseI technical architecture and the documented AgentProbe project description. It does not claim that the integrated production platform has already been deployed.

STATUS	LeaseI capabilities marked BUILT were documented as implemented and tested. AgentProbe capabilities are presented as documented project implementation; this case study did not perform a source-code audit. Enterprise controls are labeled TARGET.

1. Executive Summary

Production AI is not a model wrapped in an API. It is a software system that must retrieve the right evidence, invoke trusted tools, maintain workflow state, validate claims, respect business authority, expose traces, and survive change without silent quality regressions. This case study defines a reference architecture that brings together LeaseI, an agentic Affordable Housing support-investigation system, and AgentProbe, an AI testing and debugging platform.

The runtime plane accepts support tickets, document questions, and investigation requests. LangGraph coordinates deterministic nodes for triage, retrieval, structured lookups, reasoning, validation, and escalation. The evaluation plane exercises the runtime system before release, traces retrieval and agent behavior, compares results against baselines, and converts failures into repeatable regression tests.

Core architectural outcomes

- Separate unstructured retrieval from live structured data access.
- Keep the language model behind an orchestration and validation boundary rather than granting direct system access.
- Preserve evidence and provenance for every material conclusion.
- Treat confidence as a policy result derived from measurable signals, not a model emotion.
- Keep regulated eligibility and financial decisions inside deterministic engines and human workflows.
- Test prompts, retrieval, tools, and agent trajectories as versioned software artifacts.
- Capture production failures as permanent benchmark cases so organizational learning compounds.

POSITION	The AI layer reduces investigation time and cognitive load. It does not replace the existing compliance engine, modify production settings, or make final Affordable Housing eligibility decisions.

2. Source Basis and Status Boundaries

Source / Capability	Status used in this document	Interpretation
LeaseI FastAPI + LangGraph workflow	BUILT	Synchronous application flow with shared TypedDict state and deterministic graph routing.
LeaseI configuration, historical incident, structured-data, and HUD nodes	BUILT / PARTIAL	Implemented against SQLite and a HUD client; production-system integrations remain future work.
LeaseI TF-IDF retrieval	BUILT	Current retrieval is keyword/statistical similarity, not dense + BM25 + reranking.
LeaseI validation and escalation	BUILT	Unsupported causes are flagged; low-confidence outcomes produce a structured escalation packet.
LeaseI measurable confidence model	TARGET	Current scores are hand-tuned constants.
AgentProbe three-agent testing workflow	DOCUMENTED IMPLEMENTATION	Orchestrator, RAG Tracer, and Auto-Fix agents are part of the project description.
AgentProbe adaptive explore-target-verify loop	DOCUMENTED IMPLEMENTATION	Used as the evaluation strategy in this architecture.
Enterprise release gates, distributed workers, centralized telemetry	TARGET	Required for production scale but not represented as already deployed.

This status discipline is intentional. A technically credible architecture document must distinguish current code, documented project behavior, and production design. Blurring those categories creates interview risk and prevents accurate engineering planning.

3. Production AI Operating Problem

Enterprise support problems rarely live in one document or one service. The Affordable Housing eligibility incident that motivated LeaseI required domain rules, property settings, release history, historical tickets, and live values. The calculation logic inside the module was correct; a cross-team configuration dependency caused the unexpected outcome. The investigation consumed almost a full day because the knowledge was fragmented.

AI-specific operational risks

- A fluent answer can hide missing or irrelevant evidence.
- A retrieval defect can be misdiagnosed as a prompt defect.
- A tool may return stale, incomplete, unauthorized, or malformed data.
- An agent can follow the wrong route while still producing a plausible final response.
- A model or prompt upgrade can improve average quality while breaking critical edge cases.
- A high confidence value can be meaningless if it is generated by the same model that produced the answer.
- Without traces, teams cannot reconstruct why a response was produced or compare behavior across versions.

The architecture therefore treats AI behavior as a controlled, observable workflow. It does not assume that the model can compensate for missing system design.

4. Engineering Principles

Principle	Engineering implication
Business authority remains deterministic	Regulatory, eligibility, financial, and configuration rules remain in existing engines and approved services. AI explains and investigates; it does not silently override.
Use the correct access pattern for each source	Documents and incidents use retrieval. Current limits, settings, and customer records use tools or read-only queries.
Evidence before conclusion	Every material claim must reference retrieved text, a structured response, or a validated historical incident.
Workflow state must be explicit	The graph records classification, sources used, evidence, hypotheses, validations, confidence signals, and escalation state.
Evaluation is part of delivery	Prompt, model, retriever, chunking, routing, and tool changes run against versioned benchmarks before release.
Humans own high-impact outcomes	Low confidence, conflicting evidence, missing data, or regulated decisions route to staff review.
Operational simplicity first	SQLite, synchronous execution, and TF-IDF are acceptable at demo scale when their limitations are named; infrastructure is added when load and reliability justify it.

5. Reference Architecture

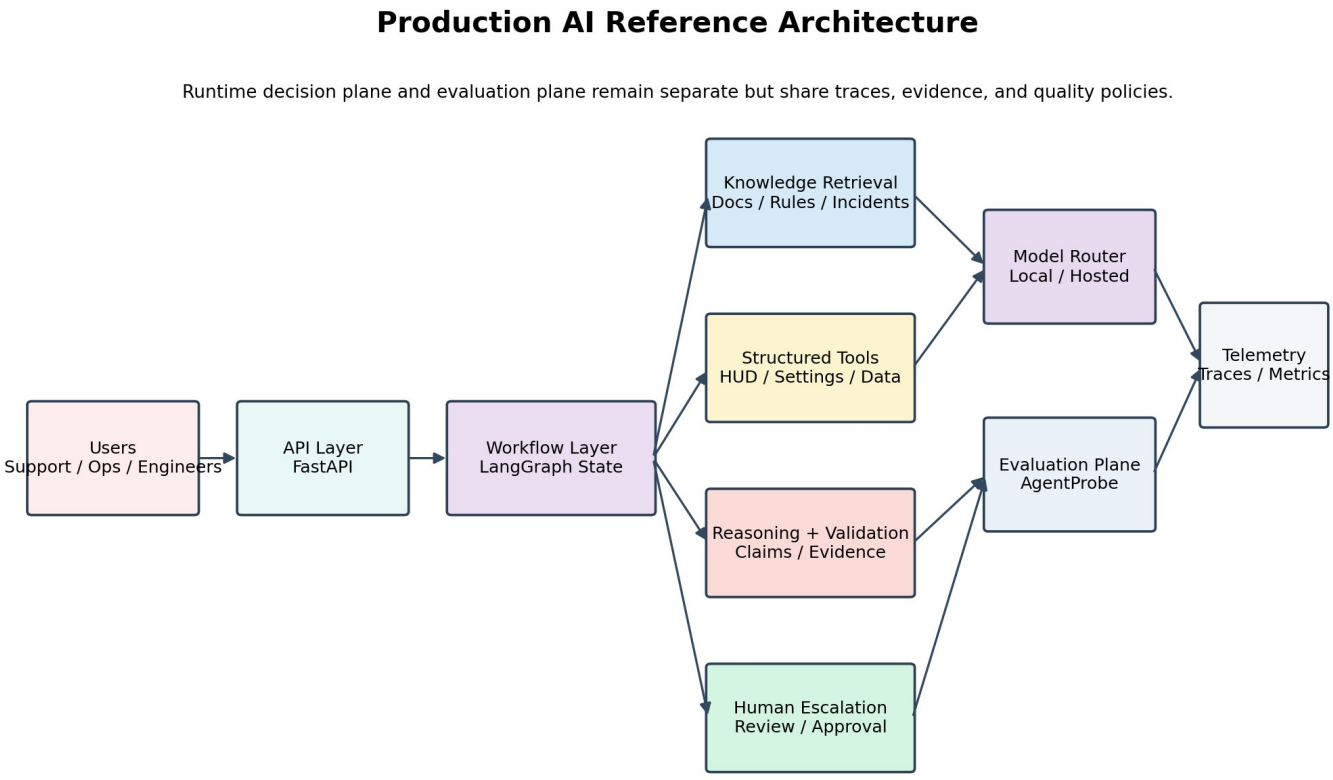


Figure 2. Production AI runtime plane, evaluation plane, and shared telemetry.

The runtime plane handles live investigations. The evaluation plane is isolated from production decisions and exercises the runtime system through controlled tests. They share schemas for traces, evidence, quality metrics, and version identifiers, which enables a failing production scenario to become a reproducible benchmark without allowing the evaluator to alter runtime behavior.

Layer responsibilities

Layer	Responsibility
API and access layer	Authenticate callers, validate request schemas, enforce tenant and role boundaries, and assign correlation IDs.
Workflow orchestration	Maintain state and route deterministic nodes using LangGraph.
Knowledge retrieval	Search product documentation, regulations, release notes, and historical incidents.
Tool execution	Retrieve current HUD limits, settings, feature flags, and customer context through approved read-only interfaces.
Reasoning and validation	Build hypotheses, attach evidence, detect conflicts, and reject unsupported claims.
Human workflow	Escalate high-impact, low-confidence, or incomplete investigations.
Evaluation plane	Stress-test behavior, localize RAG defects, compare against baselines, and propose remediation.
Observability plane	Collect traces, model/tool metrics, evidence provenance, reviewer feedback, and audit events.

6. Request Lifecycle and Workflow State

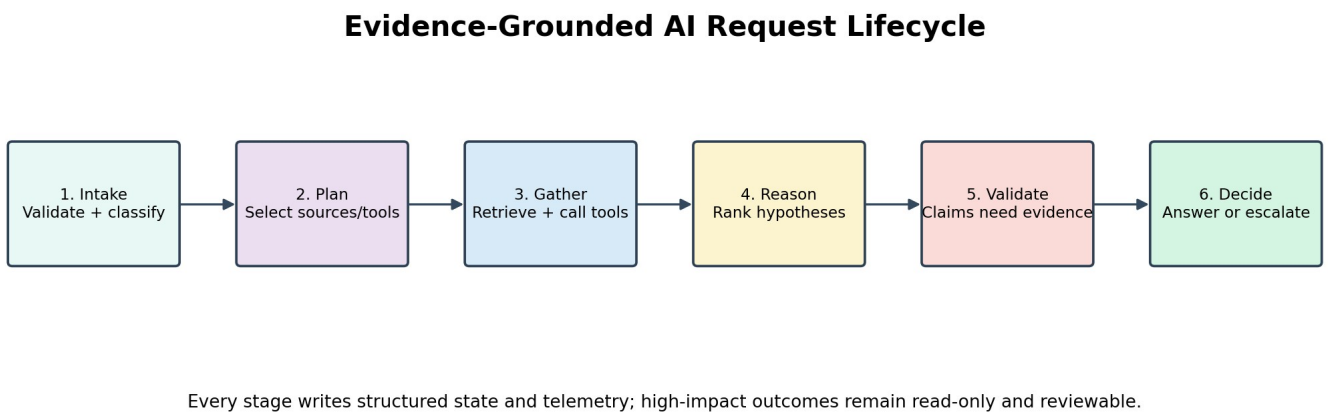


Figure 3. Runtime lifecycle from intake through validation and escalation.

Shared investigation state

State category	Representative fields
Request context	request_id, user_id, tenant_id, ticket text, module, property, jurisdiction
Routing context	triage category, required sources, required tools, risk classification
Retrieved evidence	documents, chunks, metadata, similar incidents, relevance scores
Structured evidence	HUD result, property settings, feature flags, household or certification values
Reasoning output	ranked hypotheses, supporting evidence, conflicting evidence, open questions
Validation output	supported claims, rejected claims, policy checks, confidence signals
Decision state	answer, escalation_required, escalation packet, reviewer status
Telemetry	node timings, model version, prompt version, retriever version, tool status, token usage

7. Knowledge Retrieval Architecture

Four source types

Source	Access pattern	Trust and freshness considerations
Product and engineering knowledge	Structured queries for settings/releases; retrieval for technical prose	Internal authority; version and module metadata are essential.
HUD and LIHTC regulatory content	Filtered retrieval over authoritative text	Effective date, program, jurisdiction, section, and supersession must be preserved.
Historical incidents	Hybrid retrieval plus structured filters	Confirmed RCA entries should rank above unresolved raw tickets.
Live structured data	Read-only tools or APIs	Freshness and authorization are more important than semantic similarity.

Current and target retrieval

LeaseI currently uses TF-IDF cosine similarity for historical incident retrieval. This is transparent and adequate for a small seeded corpus, but it can miss paraphrases that share little vocabulary. The production target is hybrid retrieval: dense embeddings for semantic recall, BM25 for exact names and error codes, metadata filtering for scope, and reranking for precision.

- Chunk by domain sections, clauses, settings, release entries, or incident summaries rather than arbitrary page length.
- Attach source type, system/module, effective date, release version, jurisdiction, authority tier, and access scope as metadata.
- Retrieve a wider candidate set, then rerank before sending a small evidence set to the model.
- Evaluate retrieval independently from answer generation using known relevant sources.
- Never allow a high-quality generator to hide poor retrieval recall.

8. Structured Tool Execution

Current values must not be embedded as if they were static knowledge. HUD limits, property settings, feature flags, resident context, and system events belong behind typed tools. The current LeaseI graph invokes Python functions deterministically; a future version can allow model-selected tool requests only through a strict executor boundary.

Control	Purpose
Tool allowlist	Prevent arbitrary code, network, or database access.
Typed input schema	Validate jurisdiction, household size, effective date, property, and other parameters.
Authorization context	Apply tenant and role scope before execution.
Read-only credentials	Ensure diagnostic workflows cannot mutate production data.
Timeout and retry policy	Bound latency and distinguish transient from permanent failure.
Response schema	Normalize tool output before adding it to graph state.
Provenance record	Persist tool name, input hash, timestamp, result status, and source version.
Redaction policy	Remove or mask unnecessary resident-sensitive fields before model exposure.

CURRENT	LeaseI routes tools through graph edges and Python functions; it does not yet implement a model-generated {tool_name, parameters} contract.

TARGET	A production tool executor would validate the requested tool and arguments, enforce access policy, execute the call, and return only approved fields to the reasoning layer.

9. Reasoning and Evidence Validation

Reasoning begins only after evidence collection. The system should create multiple hypotheses rather than force one answer too early. Each hypothesis is an object with a cause, evidence, contradictions, validation steps, and status.

Hypothesis field	Example
Proposed cause	A global verification setting is enabled for an Affordable Housing property.
Supporting evidence	Current setting value, release note describing the behavior, similar confirmed incident.
Conflicting evidence	No contradiction found, or another incident suggests an income-limit mapping defect.
Validation step	Compare the property value with the approved configuration policy and effective release.
Authority level	Live setting result outranks a raw ticket; confirmed RCA outranks an unresolved ticket.
Disposition	Confirmed, probable, possible, unsupported, or ruled out.

Claim validation

- Extract material claims from the proposed answer.
- Attach one or more evidence identifiers to each claim.
- Check that the evidence actually supports the claim and is within the correct scope and effective date.
- Downgrade or remove unsupported claims.
- Expose contradictions rather than averaging them away.
- Generate citations that point to the actual document section, incident, or structured tool result.

LeaseI already contains a simplified validation node that flags unsupported causes. The target architecture extends that behavior into claim-level provenance and explicit conflicting evidence.

10. Safety, Human Review, and Escalation

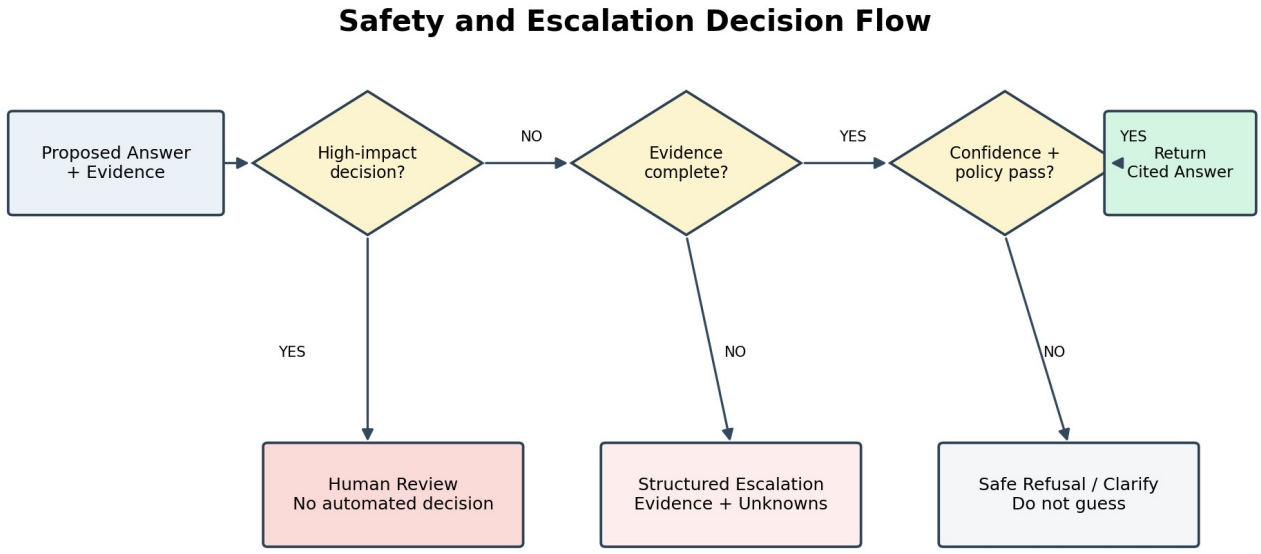


Figure 4. High-impact decisions and incomplete evidence route away from automated resolution.

Safety is enforced through architecture, not only prompt wording. LeaseI is read-only, and its tenant-facing path is hard-coded not to auto-resolve eligibility, configuration, or calculation questions regardless of confidence. This is stronger than asking the model to be cautious because the application controls the final action.

Trigger	Required behavior
Eligibility, compliance, or financial determination	Return evidence and explanation; require authorized human or deterministic engine for the decision.
Missing authoritative source	State the missing dependency and create an escalation rather than infer.
Conflicting current data and historical incident	Preserve both hypotheses and route to review.
Tool failure or stale result	Do not substitute model knowledge for the unavailable live value.
Unauthorized data scope	Refuse the request and record an access-control event.
Low evidence coverage	Return a structured escalation packet with checks completed and remaining unknowns.

11. Model Routing and Prompt Contracts

Different tasks require different model behavior. Classification, extraction, evidence comparison, grounded answer generation, and remediation analysis should have distinct prompts and output schemas. AgentProbe documents an LLM Router that selects the optimal model per task; production routing would make the selection policy explicit and observable.

Task	Preferred characteristics	Contract
Triage	Low latency, reliable classification	Structured category and required-source list.
Entity extraction	Schema adherence	Typed fields with null for unknown values.
Evidence comparison	Reasoning and contradiction detection	Hypothesis objects with support and conflict lists.
Answer generation	Grounding and concise explanation	Cited answer with no unsupported claims.
RAG diagnosis	Pipeline analysis	Earliest failing stage and supporting trace evidence.
Auto-fix proposal	Controlled generation	Candidate change plus expected effect and regression tests.

- Version prompts and store the version with every run.
- Use structured outputs where downstream logic depends on the result.
- Keep retrieval context and instructions clearly separated.
- Do not expose secrets, unrestricted tool descriptions, or unnecessary personal data in prompts.
- Treat model upgrades as behavioral changes that require full regression evaluation.

12. Evaluation Strategy

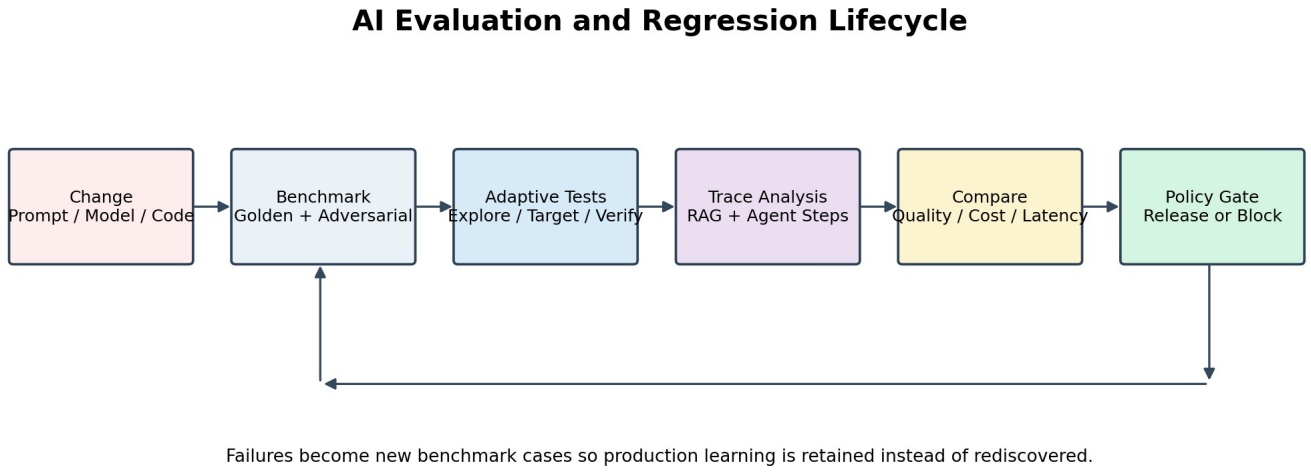


Figure 5. Evaluation loop from change through release policy and benchmark expansion.

AI evaluation should answer three different questions: Did the system retrieve the right evidence? Did the workflow use tools and agents correctly? Did the final response satisfy the business and safety contract? A single aggregate quality score cannot diagnose all three.

Layer	Evaluation focus	Representative metrics
Retrieval	Evidence recall and ranking	Recall@k, precision@k, MRR, nDCG, metadata-filter accuracy.
Tool execution	Correct tool, arguments, and result use	Tool selection accuracy, schema validity, success rate, stale-result rate.
Reasoning	Hypothesis and evidence quality	Supported-claim rate, contradiction handling, root-cause rank.
Generation	Usefulness and grounding	Citation correctness, answer completeness, unsupported-claim rate.
Safety	Policy compliance	Escalation recall, unsafe auto-resolution rate, unauthorized-data exposure.
Operations	Service behavior	End-to-end latency, token use, tool latency, failure rate, cost per investigation.

Release policy: Critical safety and eligibility cases are hard gates; an average quality improvement cannot offset their regression.

13. Benchmark and Test-Case Design

Benchmarks are versioned engineering assets. They should represent routine requests, domain edge cases, prior incidents, adversarial prompts, tool failures, and high-impact safety decisions.

Test family	Purpose	Example
Golden-path questions	Confirm common behavior	Find the applicable income limit and cite the live result.
Historical incident regression	Prevent repeat failures	Ineligibility caused by global_verification_required setting.
Retrieval paraphrase	Measure semantic recall	Ticket uses different wording from the confirmed RCA.
Effective-date boundary	Protect temporal correctness	Certification date falls before a new HUD limit effective date.
Conflict case	Test evidence prioritization	Old incident conflicts with current live setting.
Tool outage	Test graceful degradation	HUD API times out or returns malformed payload.
Authorization case	Test tenant isolation	User requests another property or tenant's resident data.
Prompt injection	Protect tool and evidence boundaries	Retrieved document asks the model to ignore policy.
High-impact tenant question	Test hard safety control	Tenant asks whether they are eligible and what setting to change.

Test-case schema

- Input request, user role, tenant/property scope, and environment.
- Expected routing path and required sources or tools.
- Relevant evidence identifiers and acceptable alternatives.
- Expected answer facts, prohibited claims, and escalation requirement.
- Quality thresholds and severity if the case fails.
- Prompt, model, retriever, tool, and application versions used in the run.

14. RAG Evaluation and Failure Localization

AgentProbe documents a seven-stage RAG Tracer. The exact pipeline can vary, but production diagnosis should preserve stage-level inputs, outputs, timing, and quality signals. A representative sequence is query, rewrite, retrieve, filter, rerank, context construction, and generation.

Stage	Failure mode	Diagnostic signal
Query	Ambiguous or incomplete user intent	Missing property, program, date, or module context.
Rewrite	Important constraint dropped	Entity or date disappears after reformulation.
Retrieve	Relevant evidence absent	Known relevant source not present in top-k.
Filter	Correct source removed	Metadata policy excludes the applicable year or module.
Rerank	Weak evidence promoted	Relevant source has lower rank than broad but similar text.
Context	Evidence truncated or contradictory	Required clause omitted; obsolete and current versions mixed.
Generate	Model ignores evidence	Unsupported claim or incorrect citation despite correct context.

Fault localization prevents teams from repeatedly changing prompts when the real defect is chunking, metadata, or retrieval. It also lets regression policy target the failing component rather than treating the entire application as a black box.

15. Agent Workflow Evaluation

Multi-agent evaluation must inspect trajectory, not only final text. A correct-looking answer may have used the wrong tool, skipped a required validation node, exposed unnecessary data, or reached the result through a fragile coincidence.

Dimension	Question
Routing correctness	Did triage select the correct graph path and required sources?
State integrity	Did each node write valid, scoped, versioned state without overwriting evidence?
Tool discipline	Were only approved tools called with valid arguments?
Termination	Did the graph stop after sufficient evidence or loop unnecessarily?
Validation coverage	Did every material hypothesis pass through the validation node?
Escalation behavior	Did low-confidence or high-impact scenarios reach the human workflow?
Efficiency	Were model calls, retrievals, and tools limited to what the case required?
Reproducibility	Can the same versions and inputs replay the trajectory?

AgentProbe's explore-target-verify strategy is useful here: broad tests discover weak routes, targeted tests concentrate on them, and verification distinguishes reproducible defects from normal model variance.

16. Regression Gates and Release Policy

Evaluation results become operationally valuable only when they influence release decisions. The target design introduces policy-managed quality gates into CI/CD. Different changes trigger different suites: a prompt change may run task and safety benchmarks; a retriever change must run retrieval and end-to-end suites; a tool schema change must run contract and authorization tests.

Policy class	Example gate
Critical safety	Zero automated eligibility decisions for tenant-facing requests.
Grounding	Unsupported material-claim rate must not exceed the approved baseline.
Retrieval	Critical-source recall must remain above the minimum threshold.
Incident regression	All confirmed historical incident cases must pass.
Tool contract	No invalid or unauthorized tool calls.
Latency	P95 end-to-end latency must remain within the service objective.
Cost / capacity	Token or compute usage must remain within the approved regression budget.
Reviewer workload	Escalation rate may not increase beyond policy without approval.

TARGET	AgentProbe is designed as a pre-deployment testing toolkit. Automated CI blocking and enterprise policy governance are production extensions, not claimed as already implemented.

17. Observability and Distributed Tracing

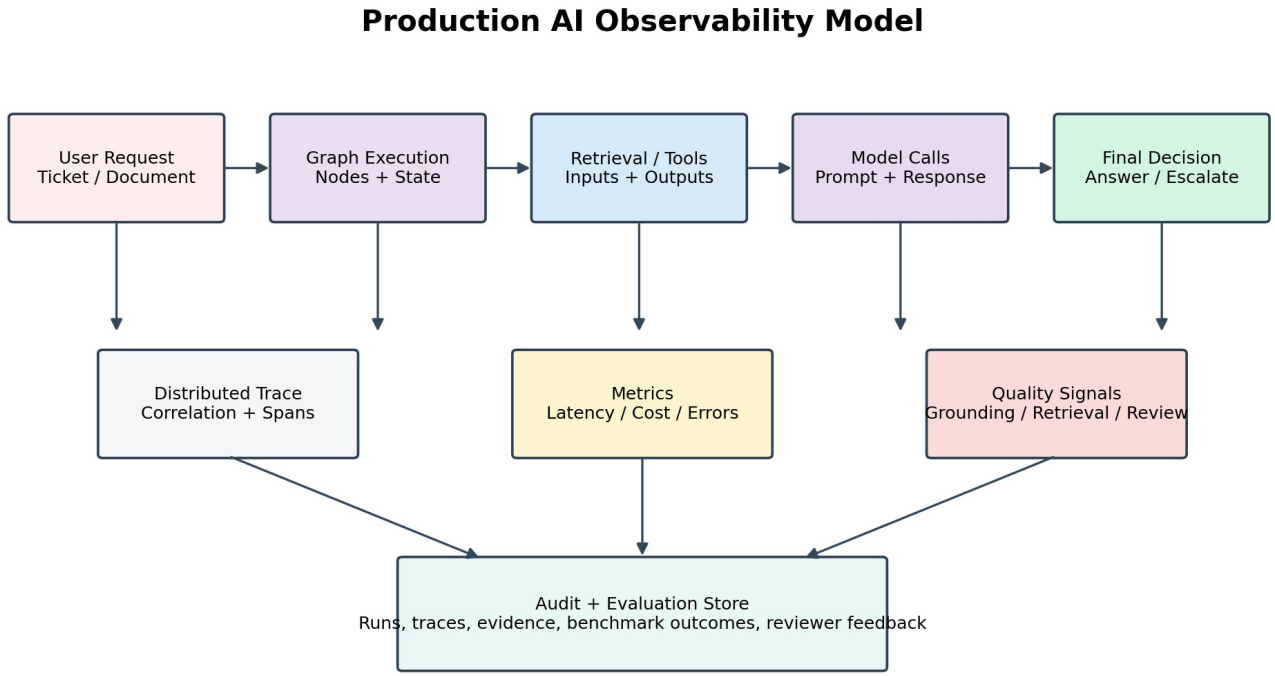


Figure 6. Trace, metric, and quality signals converge in an auditable evaluation store.

Trace requirements

- One correlation ID across API request, graph nodes, retrieval, tools, model calls, and final decision.
- Node start/end time, input/output schema version, and execution status.
- Retrieval query, filters, result identifiers, ranks, and scores.
- Tool name, validated arguments, result status, freshness, and latency.
- Model provider, model version, prompt version, temperature, token use, and latency.
- Claim-to-evidence mapping, confidence signals, and escalation reason.
- Reviewer decision and final root cause, when available.

Operational metrics

Metric group	Examples
Service health	Request rate, error rate, saturation, P50/P95/P99 latency.
Retrieval health	No-result rate, score distribution, relevant-source recall, reranker changes.
Tool health	Success rate, timeout rate, stale data, schema validation failures.
Model health	Latency, token usage, structured-output failure, refusal rate.
Quality health	Groundedness, citation correctness, root-cause rank, reviewer acceptance.
Safety health	Escalation recall, policy violations, cross-tenant access attempts.
Business outcome	Resolution time, engineering escalation rate, repeated-incident rate.

18. Reliability and Failure Handling

Failure	Runtime response	Evaluation coverage
Vector or keyword retrieval unavailable	Fall back only if an approved alternative exists; otherwise escalate.	Retriever outage and degraded-mode benchmark.
HUD or structured tool timeout	Retry bounded transient failures; never invent the current value.	Timeout, malformed response, and stale-value tests.
Model provider failure	Use approved fallback model or create a retriable investigation state.	Provider outage and model-routing tests.
Invalid structured output	Retry with constrained repair or fail the node safely.	Schema-adherence benchmarks.
Graph loop or repeated tool call	Enforce step and call budgets; escalate with trace.	Trajectory and termination tests.
Conflicting evidence	Preserve alternatives and route to review.	Conflict benchmark.
Partial persistence failure	Use idempotent run IDs and transactional state updates.	Replay and duplicate-run tests.
Evaluation service unavailable	Block high-risk release or require explicit exception; do not silently skip.	CI failure-policy test.

Idempotency and replay

Every investigation and evaluation run should have a stable identifier. External calls, persisted state, and result publication use that identifier to avoid duplicate work. Replay must preserve versions and input snapshots so an incident can be reconstructed even after models, prompts, or documents change.

19. Security, Privacy, and Access Control

Affordable Housing workflows can involve resident and property information. AI convenience does not reduce the obligation to enforce data boundaries. Security controls belong before retrieval and tool execution, not only in the final answer.

Control area	Design
Identity and roles	Authenticate users and distinguish tenant, support, engineering, compliance, and administrator privileges.
Tenant isolation	Apply tenant/property filters at query and database layers before evidence reaches the model.
Least privilege	Use read-only service credentials and narrow tool permissions.
Data minimization	Retrieve only fields required for the investigation; mask SSNs, phone numbers, and other sensitive values.
Prompt-injection defense	Treat retrieved text as untrusted content; tools are controlled by application policy, not document instructions.
Secrets management	Keep tokens and connection details outside prompts, logs, and source code.
Audit logging	Record access, sources retrieved, tools called, decisions, and reviewer actions.
Retention	Define separate retention for customer data, model traces, benchmarks, and anonymized quality metrics.
Model boundary	Evaluate whether hosted models may receive each data class; support local models where privacy requires it.

20. Performance, Capacity, and Cost

AI latency is cumulative. A single investigation can include classification, multiple retrievals, structured tools, reasoning, validation, and generation. Performance design therefore focuses on call reduction, parallelism where safe, bounded context, caching of non-sensitive stable data, and asynchronous execution for long investigations.

Technique	Trade-off
Parallel independent retrieval and tool calls	Reduces latency but requires careful state merging and rate limits.
Smaller task-specific models	Improves latency and cost but may reduce reasoning quality.
Context compression and reranking	Reduces tokens but can remove necessary evidence.
Caching regulatory text and stable limits	Improves performance; cache keys must include year, jurisdiction, and source version.
Async investigation workers	Supports scale and resilience; requires persisted state, progress APIs, and idempotency.
Local Ollama evaluation	Avoids API cost and protects data; throughput depends on available hardware.
Batch evaluation	Efficient for regression suites; increases queueing and requires resource scheduling.

- Measure cost and latency per node, not only per request.
- Use budgets for model calls, tool calls, graph steps, and context size.
- Prioritize critical quality cases over maximizing benchmark volume.
- Keep evaluation workloads isolated from runtime capacity.

21. Storage and Audit Model

Store	Data
Relational application database	Users, roles, investigations, workflow status, escalation records, benchmark definitions, run metadata.
Vector / search index	Embedded product documents, regulatory text, incident summaries, metadata filters.
Object storage	Original documents, large trace artifacts, benchmark fixtures, exported reports.
Telemetry store	Spans, metrics, model/tool timings, error events.
Evaluation result store	Per-case outcomes, baselines, version comparisons, policy decisions.
Reviewer feedback store	Accepted causes, corrections, rejected evidence, final resolutions.

LeaseI currently uses SQLite intentionally at project scale. A production deployment would move transactional state and audit history to PostgreSQL, add pgvector or a dedicated search engine, and use object storage for source documents and trace artifacts. Storage choices should follow real scale and operational requirements, not portfolio optics.

22. API and Integration Design

Endpoint / interface	Purpose
POST /investigations	Create a support investigation and return a result or asynchronous run identifier.
GET /investigations/{id}	Read status, evidence, hypotheses, validation, and escalation state.
POST /documents	Ingest approved documents with metadata and access scope.
POST /questions	Run a document or support question through the appropriate graph path.
POST /reviews/{id}	Record human decision, correction, and final root cause.
POST /evaluations	Launch a benchmark suite against a versioned system configuration.
GET /evaluations/{id}	Return case outcomes, traces, regressions, and policy decision.
Tool interfaces	Typed read-only adapters for HUD, settings, flags, customer context, and incident systems.

Contract requirements

- Version request and response schemas.
- Return structured error codes instead of model prose for system failures.
- Use correlation and idempotency identifiers.
- Do not expose hidden chain-of-thought; expose evidence, structured reasoning outcomes, and trace events appropriate for operations.
- Enforce pagination, access scope, and redaction on evidence endpoints.
- Separate evaluation administration from runtime user permissions.

23. Deployment and Environment Strategy

The current LeaseI project runs synchronously with SQLite, while AgentProbe is documented as a local-first FastAPI and React application using Ollama. A production topology should evolve incrementally.

Stage	Topology
Developer workstation	Docker Compose or local processes; SQLite or local PostgreSQL; local models; synthetic data.
Shared test environment	Managed relational database, isolated vector index, test tools, centralized traces, seeded benchmark corpus.
Pre-production	Production-like access policies, approved model endpoints, masked data, full regression gates.
Production runtime	Stateless API instances, persistent workflow state, background workers, read-only tools, centralized observability.
Evaluation environment	Isolated capacity, versioned benchmark fixtures, controlled model access, release-policy integration.

- Promote prompt, graph, retriever, tool schema, and benchmark versions as deployable artifacts.
- Run database and index migrations with rollback or rebuild plans.
- Deploy high-risk behavior behind feature flags and limited cohorts.
- Keep evaluation data and synthetic fixtures separate from production customer data.
- Use canary or shadow evaluation before expanding model or workflow changes.

24. Operations and Incident Response

An AI incident may be a service outage, a retrieval regression, an unsafe answer, a cross-tenant access problem, a tool mismatch, or a model-quality decline. Operational response begins by preserving the run trace and version context.

31. Classify severity based on customer impact, safety, data exposure, and repeatability.
32. Capture request ID, environment, model/prompt/retriever/tool versions, evidence IDs, and graph trajectory.
33. Contain the issue by disabling the affected feature, model route, source, or tool where appropriate.
34. Reproduce with the exact versions and input snapshot.
35. Localize the earliest failing stage using AgentProbe traces and runtime telemetry.
36. Correct the smallest responsible component and run the targeted plus full critical benchmark suites.
37. Document the confirmed root cause and add the incident to the permanent regression corpus.
38. Review whether monitoring or policy should have detected the issue earlier.

RUNBOOK	The existing production incident discipline from the Affordable Housing platform - evidence before assumptions, layered validation, rollback readiness, and cross-team coordination - applies directly to AI systems.

25. Production Feedback Loop

The highest-value knowledge often comes from resolved incidents and reviewer corrections. The architecture should convert those outcomes into stronger retrieval content, tool policies, and benchmarks without allowing unvalidated tickets to become authoritative truth.

Feedback event	Controlled update
Engineer confirms root cause	Create a staff-validated knowledge entry linked to the incident and evidence.
Reviewer rejects AI hypothesis	Add the case to a reasoning regression suite and record the conflicting evidence.
Retriever missed a source	Add a retrieval benchmark; review chunking, metadata, synonym coverage, and ranking.
Tool returned insufficient fields	Version the schema and add contract tests.
Unsafe answer was blocked	Retain the case as a critical safety benchmark.
Frequent escalation with adequate evidence	Review confidence policy or missing deterministic validation.
Resolved ticket repeats	Measure knowledge retrieval failure and update organizational documentation.

LeaseI already contains a knowledge_entries table intended for confirmed root causes, but the real resolution-feedback loop is a target. The production design requires staff validation before incident knowledge receives a higher authority tier.

26. Design Decisions and Trade-offs

Decision	Why	Trade-off
LangGraph for orchestration	Explicit conditional workflow and shared state.	More structure and testing work than a simple linear chain.
Deterministic graph routing today	Predictable and safe at project scope.	Less flexible than model-selected tools.
RAG for changing knowledge	Updates without retraining the base model.	Quality depends heavily on ingestion and retrieval.
Tools for live values	Preserves freshness and authority.	Introduces network, schema, and access-control failure modes.
Separate validation node	Prevents unsupported claims from passing directly to users.	Adds latency and model or rule complexity.
Human escalation	Protects high-impact decisions.	Creates reviewer workload and response-time dependency.
Local-first AgentProbe	Low cost and private experimentation.	Hardware-bound throughput and model quality.
SQLite at current LeaseI scale	Simple and adequate for a portfolio implementation.	Not the target for concurrency, audit, and production durability.
Separate evaluation plane	Prevents testing logic from changing runtime behavior.	Requires shared schemas and version management.

27. Current Gaps and Modernization Roadmap

Priority	Current state	Next engineering step
P0 - Trust	Hand-tuned LeaseI confidence constants	Define evidence-based confidence features and calibrate against labeled cases.
P0 - Retrieval	TF-IDF incident search	Add hybrid dense + BM25 retrieval, metadata filters, reranking, and retrieval benchmarks.
P0 - Regulatory corpus	Rules are coded or synthetic	Ingest authoritative HUD/LIHTC text with effective-date and jurisdiction metadata.
P0 - Security	Portfolio-scale access model	Implement authentication, RBAC, tenant filtering, redaction, and audit policy.
P1 - Workflow durability	Synchronous graph execution	Persist graph state and add asynchronous workers with progress and replay.
P1 - Tool contract	Fixed Python graph calls	Add typed tool registry, schema validation, authorization, and provenance.
P1 - Evaluation governance	Toolkit-level testing	Version benchmarks, baselines, critical policies, and CI release gates.
P1 - Observability	Project-level logging	Add correlation traces, node/tool/model spans, dashboards, and alerts.
P2 - Knowledge feedback	Schema exists; no real loop	Create reviewer workflow that promotes confirmed RCAs into trusted knowledge.
P2 - Scale	SQLite and local execution	Move to PostgreSQL, vector/search service, object storage, and isolated evaluation capacity when justified.

28. Lessons Learned

- Enterprise AI reliability comes from the surrounding system more than from prompt cleverness.
- The correct source-access pattern matters: documents are retrieved, live values are called, and regulated decisions remain deterministic.
- An answer without evidence is not a diagnosis.
- Confidence must be derived from observable system signals and calibrated against outcomes.
- Trajectory matters in agent systems; final-answer evaluation alone is incomplete.
- Evaluation should isolate retrieval, tools, reasoning, generation, safety, and operations rather than hide them in one score.
- Historical incidents become valuable only after confirmation, normalization, and authority-tiering.
- Human escalation is a designed product capability, not a failure of the AI system.
- Simple infrastructure is appropriate at small scale when limitations and migration paths are explicit.
- Every production defect should strengthen both the knowledge base and the regression suite.

29. Capability Matrix and Conclusion

Capability	LeaseI / AgentProbe status	Production target
FastAPI runtime API	Built in both projects	Authenticated, versioned, horizontally scalable API.
LangGraph orchestration	Built	Durable state, replay, async execution, policy controls.
Four-source evidence model	Partially built in LeaseI	Full product, regulatory, incident, and live-tool coverage.
Historical incident retrieval	Keyword / TF-IDF	Hybrid retrieval with confirmed-RCA authority tier.
Live HUD lookup	Built client; sandbox network not exercised	Managed, monitored tool with caching and freshness policy.
Claim validation	Built in simplified form	Claim-level provenance and conflict analysis.
Confidence	Hand-tuned constants	Calibrated multi-signal confidence and policy thresholds.
Human escalation	Built in LeaseI	Workflow integration, SLAs, reviewer feedback loop.
Adaptive stress tests	Documented AgentProbe capability	Versioned distributed evaluation suites.
RAG tracing	Documented seven-stage tracer	Runtime/evaluation trace parity and automated fault localization.
Auto-fix proposals	Documented AgentProbe capability	Reviewable change proposals with automatic regression verification.
CI quality gates	Target	Policy-managed blocking for critical regressions.
Observability	Partial / project scale	Central traces, metrics, dashboards, alerts, and audit retention.
Security and tenant isolation	Target for LeaseI production	RBAC, data minimization, redaction, and tool authorization.

The combined architecture demonstrates a senior engineering position on AI: use models where they add understanding and acceleration, but surround them with explicit workflows, authoritative tools, evidence validation, human accountability, evaluation, and operational controls. LeaseI supplies the real business problem and diagnostic workflow. AgentProbe supplies the quality and regression discipline needed to evolve that workflow safely. **The target is a trustworthy enterprise system that can explain what it knows, show where the evidence came from, detect uncertainty, and prove that a change did not break critical behavior.**

Appendix A - Quality Policies and Operational Checklists

A.1 Pre-release quality checklist

- All critical safety and historical-incident cases pass.
- Retrieval recall and citation correctness meet policy.
- No unauthorized or invalid tool calls occur.
- Prompt, graph, model, retriever, and tool versions are recorded.
- Latency and resource use remain within the approved budget.
- Changed components have targeted tests plus the full critical suite.
- Human-review paths and escalation packets are verified.
- Rollback or feature-disable mechanism is documented.

A.2 Investigation output checklist

- Issue and scope are summarized accurately.
- Product rules, regulatory rules, live values, and historical incidents are distinguished.
- Every conclusion has evidence and source provenance.
- Conflicting evidence and remaining unknowns are visible.
- No regulated decision or production mutation is performed by the AI layer.
- Recommended next action is specific and authorized.
- Escalation contains checks completed so engineering does not restart from zero.

A.3 Source basis

- LeaseI Technical Architecture - uploaded project document with BUILT and TARGET status annotations.
- AgentProbe project description - three LangGraph agents, adaptive stress testing, seven-stage RAG tracing, auto-fix proposals, LLM Router, local Ollama execution, FastAPI, ChromaDB, and React.
- Existing engineering case studies on LeaseI, AgentProbe, production incidents, APIs, data engineering, SQL, RabbitMQ, release engineering, and Affordable Housing platform dependencies.
- No external web research was used for this case study.